

The covtl-pipeline Dynamic Polar Visualization

Two-branch display of the COVTL articulatory trajectories
(vocalic background and consonantal excursions)

Frédéric Berthommier

Version 3.0 — September 2026

Scope. This manual documents the *dynamic polar figure* of the covtl-pipeline articulatory synthesizer (module `vtl_synth/video/polar_video.py`, version 3): the polar coordinate model behind it, the two-branch reconstruction of the vocalic and consonantal trajectories from the engine's own complex values, the `.polar` intermediate file, the dual-panel MP4 output, and the steps needed to integrate the feature into the GitHub repository `FBerthommier/covtl-pipeline`. It complements the main repository manual (`docs/manual.pdf`) and the language pack manual (`lang_pack/manual/manual.pdf`); the pipeline and CLI interfaces are unchanged by this feature.

Contents

1	Introduction	2
1.1	What the polar figure shows	2
1.2	Why version 3 exists	2
2	The polar coordinate model	2
2.1	Projection and targets	2
2.2	Arcs in the complex plane	3
2.3	Superposition	4
3	From engine to figure: the two-branch reconstruction	4
3.1	Instrumented re-run of the engine	4
3.2	Branch reconstruction rules	4
3.3	The <code>.polar</code> intermediate file (version 3)	5
4	Rendering the figure	5
5	Using the visualization	6
5.1	Command line	6
5.2	Python API	7
6	Worked example and validation	7

7 Integrating the visualization into the GitHub repository	8
7.1 Required pipeline adaptations: none at the interface level	8
7.2 Files to commit	9
7.3 Step-by-step	9
7.4 Caveats to document in the release notes	9

1 Introduction

1.1 What the polar figure shows

The covtl-pipeline synthesizes speech by driving VocalTractLab with articulatory trajectories computed in *polar coordinates* (ρ, θ) : every phoneme of the inventory carries a target point — a radius ρ (cm) and an angle θ (rad) in the midsagittal plane — and the trajectory engine moves an abstract articulator through these targets along curved arcs. The polar figure makes this hidden machinery visible: it replays, as an animated curve, the path the engine actually takes.

Because the model is a *superposition* model, a single path is not enough. At every instant two gestures run simultaneously:

- the **vocalic branch** $z_v(t)$ (drawn in **red**), the vowel-to-vowel background that carries the syllable nucleus, moving from anchor to anchor;
- the **consonantal branch** $z_c(t)$ (drawn in **blue**), which exists only inside consonant clusters: it *detaches* from the vocalic background, visits the consonant targets $C_1, \dots, C_m$ one after another, and returns to the vocalic path.

Figure 1 shows the two branches over the whole utterance **ba.da.ga**: the red background sweeps between the vowel anchors while the blue excursions leave it for /b/, /d/ and /g/ and come back. During each cluster the two curves are *simultaneous and dissociated* — the defining property of the display.

1.2 Why version 3 exists

Earlier versions reconstructed the displayed path by *inverting* the 15 parameter frames actually sent to the synthesizer (a least-squares fit of the polar projection). That single inverted path cannot represent the superposition: inside a cluster the parameter frames are a blend of two gestures and lie *off* the polar manifold, so the fit zigzags and the two branches collapse into one visible path. Version 3 abandons the inversion on the rendering path and rebuilds both branches *directly from the engine's own polar points*, using the exact arc machinery of the reference model (Section 3). The inversion function `polar_from_pval` is still shipped, as a diagnostic tool only.

2 The polar coordinate model

2.1 Projection and targets

The polar projection maps a target (ρ, θ) onto the synthesizer parameter space. For every parameter p of the 15 COVTL parameters, with calibration constants $c_0^{(p)}, c_1^{(p)}, c_2^{(p)}$ (columns of the matrix `C0`):

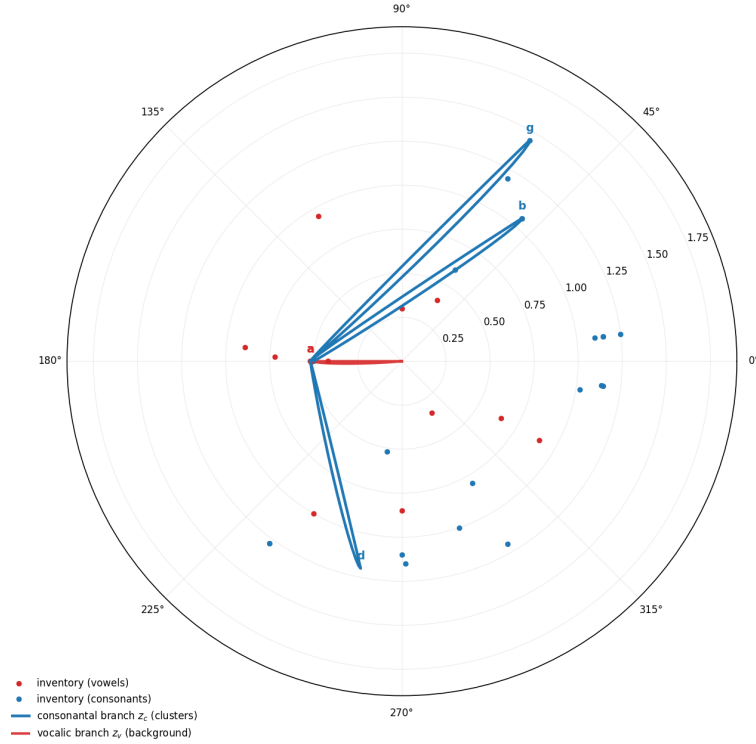


Figure 1: Static view of the two branches for **ba.da.ga** (JD3 speaker, internal notation). Red: the vocalic background $z_v(t)$, always active. Blue: the consonantal branch $z_c(t)$, active only during the three clusters; each blue excursion peaks exactly on the consonant target (large dots: phoneme inventory; small labels: phoneme keys at their targets).

$$P_p(\rho, \theta) = c_1^{(p)} + \rho c_0^{(p)} \cos(c_2^{(p)} - \theta). \quad (1)$$

Vowel targets come from the speaker calibration (**VOWEL_TARGETS**, e.g. /a/ at $\rho = 0.52$, $\theta = \pi$ for JD3); consonant targets from the consonant table plus contextual place rules (dental /d/, velar /g/ resolved by vowel context, cluster-specific adjustments). In the complex plane a target is the complex number $z = \rho e^{i\theta}$.

2.2 Arcs in the complex plane

All motion is produced by one primitive, `polar.polar_arc`, which interpolates between a departure point $z_1 = \rho_1 e^{i\theta_1}$ and an arrival point $z_2 = \rho_2 e^{i\theta_2}$. Writing th for a half-turn sweep from 0 to π (or π to 0 when `orientation='inverse'`, which is the departure-to-arrival direction used by the engine and the display) and $r_k = \cos^2(th/2)$:

$$z(th) = (1 - r_k) z_1 + r_k \rho_2 e^{i(\theta_2 + \frac{\nu}{K} th)}. \quad (2)$$

Three parameters control the shape of the path:

- **Curvature K .** The arrival angle carries an extra phase ν/K ; the smaller K , the more the arc bows away from the straight chord. The display uses $K = K_{\text{VOWEL_DEFAULT}} = 30$ (Berthommier v15g reference), which gives a visible but moderate curvature (sagitta 2–2.4 % of the chord on the **ba.da.ga** validation corpus). Note that the trajectory *engine* drives its

parameter arcs with $K = 1000$ (quasi-rectilinear); using $K = 30$ on screen is a deliberate visualization choice that restores the geometry of the reference model.

- **Direction ν .** $\nu = +1$ bows the vocalic arcs one way, $\nu = -1$ the consonantal sub-arcs the other way, as in the reference CV superposition figure.
- **Angular path.** The arrival angle is unwrapped so the arc always takes the shortest angular route (no $-\pi \leftrightarrow +\pi$ wrap-around jumps).

Stationary holds (vowel plateaus, decay/attack anchors) are constant complex samples produced by `polar.stationary_point`. Both primitives sample at 100 Hz, the gesture rate of the engine (1 step = 10 ms).

2.3 Superposition

At every instant the final parameter frame sent to the synthesizer is a blend: the *selector* parameters owned by the active consonants follow the consonantal branch, while all other parameters follow the vocalic background (function `build_cluster_pval` of `syltraj.py`). The figure mirrors this architecture by keeping the two branches separate on screen instead of merging them into one path.

3 From engine to figure: the two-branch reconstruction

3.1 Instrumented re-run of the engine

The figure must show *exactly* the timing of the synthesis. The pipeline therefore re-runs the same engine function (`syltraj.build_phrase_pval`, with the same normalization and the same duration parameters) while temporarily wrapping the five helpers of `build_global_pval` that carry the polar geometry. Each helper records, per appended block and in `block_info` order, the polar points the engine drives toward:

Recorded event	Content
<code>arc</code>	departure and arrival anchor pair + duration in steps (pause, initial and terminal arcs)
<code>bg</code>	anchor pair + duration (vocalic background arcs; the schwa routing of closing VV arcs records its two half-arcs)
<code>hold</code>	anchor point + duration (decay/attack stationary holds)
<code>cluster</code>	anchor pair, list of consonant <i>node</i> targets $[(\rho_k, \theta_k)]$ (context-adjusted), sub-arc duration T

Vowel plateau blocks are appended inline by the engine (no helper to wrap); the reconstruction rebuilds them from `VOWEL_TARGETS[vowel_key]`. After the run, every queue must be exactly consumed by a pass over `block_info`; any leftover event or duration mismatch raises a `RuntimeError` (alignment drift guard), so a silent desynchronization of the figure with respect to the audio is impossible by construction.

3.2 Branch reconstruction rules

The pass over `block_info` emits, at 100 Hz, the two aligned trajectories:

Block kind	Branch behavior
plateau	red: stationary vowel target; blue: —
background, pause, initial, terminal	red: arc(<i>dep</i> , <i>arr</i>) with $K=30$, $\nu=+1$; blue: —
decay, attack	red: stationary anchor; blue: —
cluster, $n = (m+1)T$ steps	red: arc(<i>dep</i> , <i>arr</i>) over the whole block (the background keeps flowing anchor to anchor); blue: sub-arcs $dep \rightarrow C_1 \rightarrow \dots \rightarrow C_m \rightarrow arr$, T steps each, $K=30$, $\nu=-1$

Two properties deserve emphasis. First, *during a cluster the red branch is not interrupted*: the vocalic background runs underneath the consonant excursion (this is the **bg** component of **build_cluster_pval**). Second, the blue branch peaks *exactly* on the consonant node targets at the end of each approach sub-arc — on the validation corpus **ba.da.ga** the reconstructed peaks coincide with the engine targets to within 10^{-9} (Table 1).

3.3 The .polar intermediate file (version 3)

The result is stored in a self-documented JSON file with `format = covtl-polar` and `version = 3`:

```
{
  "format": "covtl-polar", "version": 3,
  "sample_rate": 100, "speaker": "JD3",
  "n_steps": 176, "duration_s": 1.76,
  "columns": { "vocalic_rho": "...", "consonantal_theta": "...", ... },
  "orientation": "trigonometric: theta=0 East, counter-clockwise",
  "trajectory": {
    "vocalic": { "rho": [...], "theta": [...] },
    "consonantal": { "rho": [null, ..., 0.96, ...], // null = inactive
                    "theta": [null, ..., 0.87, ...] }
  },
  "phonemes": [ { "key": "b", "category": "C", "rho": 1.06,
                  "theta": 0.87, "t0": 20, "t1": 36 }, ... ],
  "inventory": [ { "key": "a", "category": "V", ... }, ... ]
}
```

The two 100 Hz trajectories are the single source of the figure — the video never re-derives them from the **.tract**. The consonantal arrays carry `null` while the branch is inactive (outside clusters), which lets the renderer break the blue curve cleanly. Phoneme timing is in gesture steps (10 ms per step); `t0/t1` are exact multiples of the block boundaries, so the **.polar**, **.tract** and **.wav** timelines agree by construction ($n_{\text{tract frames @400 Hz}} = 4 \times n_{\text{steps}}$). Files of version 2 (single blended trajectory plus per-step categories) remain readable: the loader maps them conservatively onto the two-branch structure.

4 Rendering the figure

The final MP4 is a dual-panel composition at 25 Hz (H.264 + AAC, audio muxed from the synthesized **.wav**):

- **Left panel**: the standard sagittal tract view, rendered from the **.tract** file (unchanged by this feature).
- **Right panel**: the polar figure, in trigonometric orientation ($\theta = 0$ East, counter-clockwise), with: the two branch curves; a fading trail of ≈ 0.75 s per branch (red and blue independently); a current-position dot per active branch; the phoneme inventory as small permanent

dots (red vowels, blue consonants); a thin segment from the current position toward the active target; a transient label of the current phoneme (internal input notation, e.g. `tS`, `a~`) displayed 0.5 s from the onset of the phoneme near its target.

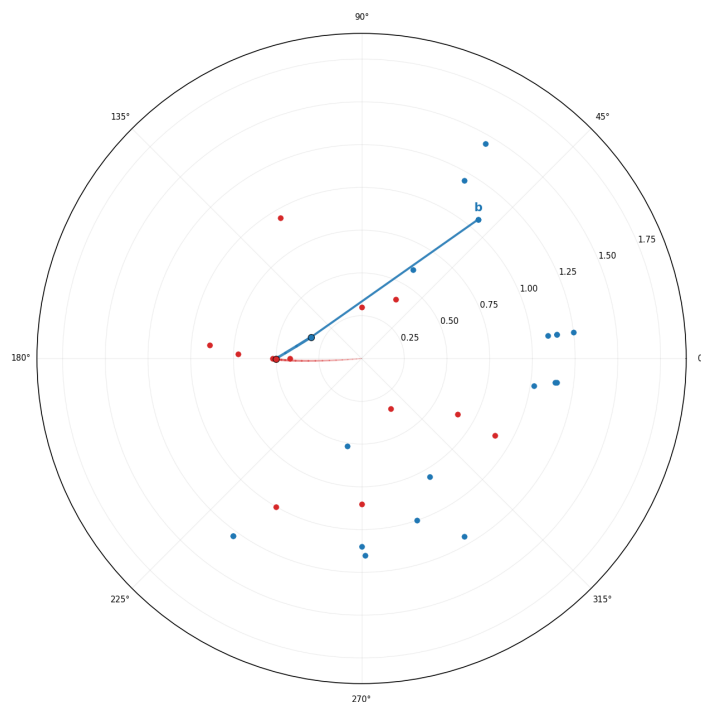


Figure 2: Rendered frame at $t = 0.24$ s in `ba.da.ga`, during the `/b/` cluster: the blue branch has detached from the red background and reaches the `/b/` target (upper right, $\rho = 1.06$, $\theta \approx 0.87$ rad), while the red background continues its progression toward the vowel anchor.

5 Using the visualization

5.1 Command line

The feature is carried by three subcommands of `vtl-synth`; the interfaces are identical to the rest of the pipeline:

```
# Full chain: .tract + .wav + .polar + dual-panel .mp4
vtl-synth run --polar "this is easy for us" -l en -o out --label polar_demo_en

# .polar only, from a phrase in the internal notation
vtl-synth polar-build "ba.da.ga" -o out/ba.polar --tcons 160

# Dual-panel video from existing files
vtl-synth polar-video out/polar_demo_en.tract --polar out/polar_demo_en.polar \
    out/polar_demo_en.wav -o out/polar_demo_en.mp4
```

Reproducing the reference demos requires the **JD3 speaker** to be the active speaker (`python install_speaker.py install jd3`; restore your previous speaker afterwards). The engine options (`-l`, `-tcons`, `-tvoy`, `-tpause`, `-f0`, `-s`) behave exactly as in `run` without `-polar`, because the `.polar` build re-runs the same engine with the same parameters.

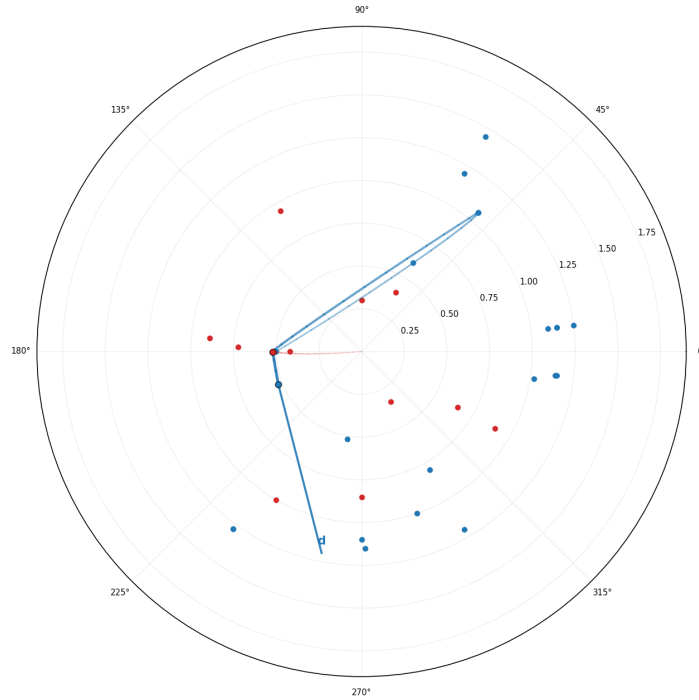


Figure 3: Rendered frame at $t = 0.64$ s, during the /d/ cluster: the fading blue trail of the /b/ excursion is still visible (memory ≈ 0.75 s) while a new blue excursion approaches the /d/ target (bottom, $\rho = 1.2$, $\theta \approx -1.77$ rad); the red background stays on the vowel axis. The two branches are simultaneous and dissociated.

5.2 Python API

```
from vtl_synth.video.polar_video import (
    build_polar, write_polar, read_polar,
    render_polar_frame, make_polar_video)

# 1. Build the two-branch trajectories (timings of the synthesis)
pol = build_polar(["ba.da.ga"], t_cons=16, t_voy=8,
                  pause_short_ms=200.0, pause_long_ms=320.0)
write_polar("out/ba.polar", pol)

# 2. Read it back (v3 or legacy v2 files)
pol = read_polar("out/ba.polar")
zv, zc = pol["traj_v"], pol["traj_c"]  # (n, 2) rho/theta, NaN = inactive

# 3. Draw one frame (matplotlib polar axes) or the whole MP4
import matplotlib.pyplot as plt
fig = plt.figure(); ax = fig.add_subplot(111, projection="polar")
render_polar_frame(ax, pol, t=0.64)
make_polar_video("out/ba.tract", "out/ba.polar", wav_path="out/ba.wav",
                 out_path="out/ba.mp4")
```

6 Worked example and validation

The validation corpus is `ba.da.ga` with $T_{\text{cons}} = 16$ steps (160 ms), $T_{\text{voy}} = 8$ steps, pauses 200/320 ms, JD3 speaker. Table 1 compares the reconstructed blue-branch peaks with the engine consonant targets; Figure 1 (page 3) shows the full utterance.

Automated checks:

Table 1: Blue-branch peaks vs. engine consonant targets on `ba.da.ga` (JD3).

Consonant	Engine target (ρ, θ)	Reconstructed peak	Error	Sagitta (% chord)
/b/	(1.06, 0.87)	(1.060, +0.873)	$< 10^{-9}$	2.0
/d/	(1.20, -1.77)	(1.200, -1.767)	$< 10^{-9}$	2.4
/g/	(1.45, +1.05)	(1.450, +1.047)	$< 10^{-9}$	2.2

- **Targets.** every blue sub-arc terminates exactly on its node target (angle compared modulo 2π).
- **Simultaneity.** the blue branch is active on every step of each cluster block $((m+1)T$ steps: approach *and* release), and only there.
- **Background continuity.** the red branch is finite on every step of the utterance and rests exactly on the vowel target during plateaus.
- **Curvature.** the sagitta of every sub-arc is between 2 % and 2.4 % of its chord ($K = 30$; the engine's $K = 1000$ would give ≈ 0.07 %, invisible).
- **Timeline.** `.polar` duration = `.tract` duration = MP4 duration (within one 25 Hz frame); the test selection `pytest tests -k "video or pipeline or cli or polar"` passes unchanged (9 tests).

7 Integrating the visualization into the GitHub repository

This section is a hand-over recipe: what to commit to `FBerthommier/covtl-pipeline`, what must (and does not must) be adapted, and how to keep the repository coherent.

7.1 Required pipeline adaptations: none at the interface level

The feature was designed to be *additive*. No signature changed:

- `Pipeline.run(...)` keeps its signature; the polar figure is activated by the existing keyword `polar=True`, which already existed for v2 and now writes a version 3 file.
- The CLI subcommands (`run -polar`, `polar-build`, `polar-video`) keep their arguments.
- `vtl_synth/core/polar.py` (the model primitives) and `vtl_synth/core/syltraj.py` (the engine) are *not modified*: the reconstruction only *reads* them (temporary monkeypatching, restored in a `finally` block), so the synthesis and the regression baselines are untouched.

The only adaptation *inside* the repository is declarative: `matplotlib` must appear among the dependencies (it was already optional for `plot-tract`; make it a hard dependency of the video extra, or of the core install, since `run -polar` now requires it). In `pyproject.toml`:

```
[project.optional-dependencies]
plot  = ["matplotlib"]
video = ["Pillow", "imageio-ffmpeg", "matplotlib"]
```

7.2 Files to commit

Path	Content / action
vtl_synth/video/polar_video.py	the v3 module (modified)
docs/polar_visualization.tex/.pdf	this manual (new)
docs/figures/fig_.png	the three figures above (new)
CHANGELOG.md	the v3 entry under [Unreleased] (modified)
tests/	optional: pin the v3 invariants (targets, blue-span activity, round-trip, v2 readability) as unit tests

The demo *media* (out/polar_demo_en.mp4, ≈ 0.3 MB each, plus .tract/.wav/.polar) may be committed as small showcase files or regenerated on demand; if you want them in the repository, prefer **Git LFS** for the MP4s (`git lfs track "*.mp4"`) or add a `scripts/make_demos.py` that regenerates them (see below). The `out/` directory is otherwise a working directory and may stay ignored.

7.3 Step-by-step

```
# 1. Branch
git checkout -b feature/polar-video-v3

# 2. Module + dependencies
git add vtl_synth/video/polar_video.py pyproject.toml

# 3. Documentation and figures
git add docs/polar_visualization.tex docs/polar_visualization.pdf
    docs/figures/

# 4. Optional demo regeneration script + CHANGELOG
git add scripts/make_demos.py CHANGELOG.md

# 5. Verify, then push
pytest tests -k "video or pipeline or cli or polar"      # 9 passed
git commit -m "polar video v3: dissociated vocalic/consonantal branches"
git push -u origin feature/polar-video-v3
```

A minimal `scripts/make_demos.py` regenerates both showcase videos (JD3 must be the active speaker):

```
from vtl_synth import Pipeline
for label, kwargs in [
    ("polar_demo_en", dict(text="this is easy for us", use_g2p=True)),
    ("polar_e2e", dict(text="ba.da.ga", use_g2p=False)),
]:
    Pipeline(f0_hz=102.216, speaker="JD3", t_cons_ms=100.0,
             t_voy_ms=80.0).run(
        kwargs["text"], output_dir="out", use_g2p=kwargs["use_g2p"],
        label=label, polar=True)
```

7.4 Caveats to document in the release notes

- **Speaker dependence.** the figure uses the polar targets of the *active* speaker constants. The reference demos are JD3; other speakers (s1, s2, m01, w02) work but their targets differ (e.g. /b/ is (0.96, 4.90) for s2).

- **Speaker switching.** generating the demos requires installing JD3 (`install_speaker.py`), which swaps the shared wheel resource; restore the previous speaker afterwards.
- **.polar versioning.** readers of v2 files must migrate to `read_polar`, which accepts both versions.
- **Display curvature.** $K = 30$ is a visualization constant (reference-model geometry); the parameter arcs of the engine remain $K = 1000$. This is intentional and documented in the module header.

Reference model: F. Berthommier, “COVTL” technical reports v15g, arXiv:2307.02299 (2023) and JEP 2024, pp. 541–550.